Programowanie Usług Sieciowych

Zajęcia projektowe

Przebieg lab:

- 2 małe projekty indywidualne (zapoznanie się z technologią)
- 1 projekt grupy rozbity na 3 etapy (grupa 2-3 studentów)

Ocena końcowa: średnia ważona z 2 małych projektów i projektu grupowego:

$$\text{ocena} = (P1 + P2 + 3*PG) / 5$$

Termin: podany na Delta w module uploadu rozwiązania

Spóźnienie: pół oceny lub 5pkt w dół za każdy rozpoczęty tydzień opóźnienia, ale nie mniej niż do odpowiednio 3.0 lub 50%

Projekt 1: starter pack do programowania aplikacji sieciowych klienta i serwera

- na ocenę 3.0: TCP, UDP, IPv4
- na ocenę 3.5: IPv6
- na ocenę 4.0: gniazda surowe
- na ocenę 4.5: uprawnienia Linux - gniazda surowe i serwer na portach <1024 bez uprawnień root'a
- na ocenę 5.0: konfiguracja firewall'a do przepuszczania/blokowania ww. ruchu

We wszystkich ww. przedmiotem jest implementacja zarówno klienta jak i serwera. Technologia realizacji dowolna. Na wykładzie przedstawione będzie jak to zrobić w języku C bezpośrednio na bibliotece UNIX Socket. Może być Python, Java etc. liczy się to by osobiście zaprogramować komunikację w warstwie 4 (TCP, UDP) i 3 (gniazda surowe), nie przez jakąś bibliotekę.

Forma pracy: kod źródłowy + prezentacja jak to działa + pytania kontrolne sprawdzające czy praca jest samodzielna.

Projekt 2: bezpieczna komunikacja

- na ocenę 3.0: klient i serwer TCP wykorzystujący komunikację szyfrowaną TLS na dowolnych certyfikatach SSL (np. self signed)
- na ocenę 3.5: własne CA, konfiguracja, generowanie certyfikatów SSL, wykorzystanie w komunikacji,
- na ocenę 4.0: uwierzytelnianie certyfikatem SSL obu stron.
- na ocenę 4.5: klient QUIC, nowoczesny protokół który powoli zastępuje TCP, zwłaszcza w zastosowaniach dla internetu mobilnego,
- na ocenę 5.0: serwer QUIC

Dozwolone metody wykonania podobne jak w przypadku projektu 1. Na wykładzie będzie przedstawione jak to zrobić w bibliotece OpenSSL 3.5 LTS.

Forma pracy: kod źródłowy + prezentacja jak to działa + pytania kontrolne sprawdzające czy praca jest samodzielna.

Projekt grupowy 2-3 osoby:

Proponowany podział prac: jedna osoba pracuje nad serwerem, druga osoba pracuje nad klientem, trzecia osoba odpytania kontrolne sprawdzające czy praca jest samodzielnapowiada za to, że całość będzie działać i dokumentacja będzie poprawnie przygotowana.

Forma pracy: jeden kod źródłowy, jedna dokumentacja i jedna prezentacja na zespół.

Uwaga! wszyscy członkowie zespołu wrzucają ww. materiały na delta, jest to wymóg wpisania oceny na platformie Delta.

Wymagania techniczne

- Komunikacja sieciowa musi używać zaprojektowanego przez studentów protokołu aplikacyjnego (własny format komunikatów, stany, kody błędów itp.).
- Protokół ma uwzględniać:
    - bezpieczną komunikację (np. TLS, podpisy wiadomości, HMAC, tokeny sesyjne, uwierzytelnienie),
    - obsługę błędów połączenia (timeout, reconnect, błędne pakiety, utrata połączenia, duplikaty, częściowe dane itp.).

Dopuszczalne są biblioteki kryptograficzne / TLS (np. OpenSSL).

Etap 1: Projekt autorskiego protokołu sieciowego

Projekt w postaci dokumentu PDF zawierający:

1. Cel protokołu i zakres
    a. Do czego służy protokół?
    b. Jakie problemy rozwiązuje?
    c. W jakim modelu działa (client–server / P2P)?
2. Założenia techniczne
    a. Transport: TCP / UDP / QUIC / WebSocket / inny
    b. Kodowanie wiadomości: tekstowe (JSON, CSV, własny format) lub binarne
    c. Założenia dotyczące niezawodności (co zapewnia warstwa transportowa, a co protokół aplikacyjny)
3. Struktura komunikatów
    a. typy wiadomości (np. HELLO, AUTH, DATA, ACK, ERROR, PING, BYE),
    b. pola wiadomości (wymagane/opcjonalne),
    c. format (przykładowe ramki),
    d. walidacja (co jest błędem formatu).
4. Model stanów / przebieg komunikacji
    a. opis sesji: nawiązanie połączenia → autoryzacja → wymiana danych → zamknięcie,
    b. diagram stanów lub diagram sekwencji,
    c. zasady timeoutów, retransmisji/retry, keep-alive.
5. Bezpieczeństwo

    a. jak realizowane jest bezpieczeństwo komunikacji:
        i. poufność (np. TLS),
        ii. integralność (np. HMAC/podpis),
        iii. uwierzytelnienie (login+hasło, token, klucz API),
        iv. autoryzacja (jeśli dotyczy),
        v. ochrona przed replay (np. nonce/timestamp/ID wiadomości).
        vi. krótki model zagrożeń (przynajmniej podstawowy).
6. Obsługa błędów i awarii połączenia
    a. kody błędów i ich znaczenie,
    b. zachowanie po błędach składni/protokołu,
    c. timeout połączenia,
    d. utrata połączenia w trakcie sesji,
    e. duplikaty/niekompletne wiadomości,
    f. limity i ochrona przed nadużyciami (np. rate limit, max rozmiar komunikatu).
7. Przykładowe scenariusze komunikacji
    a. minimum 2–3 kompletne przykłady (poprawna sesja + przypadek błędu).

Kryteria oceniania – ETAP 1 (30 pkt)

A. Kompletność i spójność specyfikacji (8 pkt)
    a. 0–2 pkt: opis fragmentaryczny, brak kluczowych elementów
    b. 3–5 pkt: większość elementów obecna, ale niespójności
    c. 6–8 pkt: pełna, spójna specyfikacja możliwa do implementacji
B. Bezpieczeństwo komunikacji (8 pkt)
    a. 0–2 pkt: bezpieczeństwo potraktowane hasłowo
    b. 3–5 pkt: wskazano mechanizmy, ale brak uzasadnień / luk
    c. 6–8 pkt: sensowny model bezpieczeństwa, poprawne użycie standardowych mechanizmów
C. Obsługa błędów i awarii połączenia (6 pkt)
    a. 0–2 pkt: brak lub szczątkowy opis
    b. 3–4 pkt: podstawowe błędy uwzględnione
    c. 5–6 pkt: dobrze zdefiniowane błędy, timeouty, recovery
D. Formalizacja protokołu (formaty, stany, sekwencje) (5 pkt)
    a. 0–1 pkt: brak formalnego opisu
    b. 2–3 pkt: częściowy opis
    c. 4–5 pkt: czytelny format, stany i przebiegi komunikacji
E. Jakość dokumentacji i przykładów (3 pkt)
    a. 0–1 pkt: mało czytelne, brak przykładów
    b. 2 pkt: czytelne, przykłady częściowe
    c. 3 pkt: bardzo czytelne, dobre przykłady poprawnych i błędnych scenariuszy

Etap 2: Projekt aplikacji wykorzystującej protokół (klient–serwer lub P2P) z etapu 1

Projekt w postaci dokumentu PDF zawierający:

1. Opis funkcjonalny aplikacji
    a. Co aplikacja robi?
    b. Jaki problem użytkownika rozwiązuje?
    c. Kto jest użytkownikiem (aktorzy)?

2. Architektura rozwiązania
    a. model: klient–serwer / P2P,
    b. komponenty (np. klient GUI/CLI, serwer, baza danych, moduł autoryzacji),
    c. diagram komponentów / wdrożenia,
    d. przepływ danych między komponentami.
3. Przypadki użycia (use cases)
    a. minimum 4–6 przypadków użycia (w zależności od skali projektu),
    b. dla każdego:
        i. cel,
        ii. aktor,
        iii. warunki wstępne,
        iv. scenariusz główny,
        v. scenariusze alternatywne / błędy,
        vi. wynik końcowy.
    c. przypadki użycia mają udowadniać poprawność projektu aplikacji (czyli że zaprojektowany system rzeczywiście realizuje wymagania).
4. Mapowanie aplikacji na protokół
    a. które funkcje aplikacji korzystają z których komunikatów protokołu,
    b. jak protokół wspiera przypadki użycia,
    c. ewentualne rozszerzenia protokołu (z uzasadnieniem).
5. Wymagania niefunkcjonalne
    a. bezpieczeństwo,
    b. wydajność (np. liczba klientów, opóźnienia),
    c. niezawodność,
    d. skalowalność (choćby podstawowa),
    e. logowanie / diagnostyka.
6. Plan implementacji i testowania
    a. zakres MVP (minimum działające),
    b. plan testów (funkcjonalne, błędów, bezpieczeństwa, obciążeniowe – choćby podstawowe),
    c. podział pracy (dla zespołu).

Kryteria oceniania – ETAP 2 (25 pkt)

A. Poprawność i sensowność koncepcji aplikacji (5 pkt)
    a. 0–1 pkt: pomysł niejasny / nieadekwatny
    b. 2–3 pkt: pomysł poprawny, ale mało rozwinięty
    c. 4–5 pkt: sensowna, dobrze uzasadniona aplikacja z realnym use-case
B. Projekt architektury (6 pkt)
    a. 0–2 pkt: architektura chaotyczna lub nieopisana
    b. 3–4 pkt: architektura poprawna, ale powierzchowna
    c. 5–6 pkt: czytelna architektura, logiczny podział komponentów i przepływów
C. Przypadki użycia i scenariusze (6 pkt)
    a. 0–2 pkt: mało przypadków użycia lub słabo opisane
    b. 3–4 pkt: poprawne, ale brak alternatyw/błędów
    c. 5–6 pkt: kompletne, dobrze opisane, obejmują scenariusze błędne i potwierdzają poprawność projektu
D. Zgodność projektu aplikacji z protokołem (5 pkt)

    a. 0–1 pkt: brak powiązania
    b. 2–3 pkt: częściowe powiązanie
    c. 4–5 pkt: jasne mapowanie funkcji aplikacji na komunikaty/stany protokołu
E. Plan implementacji i testów (3 pkt)
    a. 0–1 pkt: bardzo ogólny
    b. 2 pkt: podstawowy i użyteczny
    c. 3 pkt: konkretny plan MVP + testów + kryteriów zakończenia

Etap 3: Implementacja aplikacji

Projekt w postaci kodu źródłowego i dokumentacji uruchomienia oraz prezentacja działania.

Minimalne wymagania implementacyjne

1. Działająca komunikacja zgodna z protokołem
    a. klient–serwer lub P2P działa w praktyce,
    b. poprawna obsługa komunikatów,
    c. walidacja danych wejściowych.
2. Bezpieczeństwo (praktycznie)
    a. wdrożenie zadeklarowanych mechanizmów (np. TLS / tokeny / HMAC),
    b. bezpieczna obsługa błędów (bez wycieku wrażliwych informacji),
    c. podstawowe zasady bezpieczeństwa w kodzie.
3. Obsługa błędów i odporność
    a. timeouty,
    b. rozłączenia,
    c. błędne komunikaty,
    d. sensowne logi / komunikaty diagnostyczne.
4. Testy
    a. testy funkcjonalne (manualne lub automatyczne),
    b. testy scenariuszy błędnych,
    c. krótkie testy wydajnościowe/obciążeniowe (jeśli sensowne dla projektu).
5. Dokumentacja uruchomienia
    a. instrukcja uruchomienia,
    b. wymagania środowiskowe,
    c. przykładowe komendy / konfiguracja,
    d. opis znanych ograniczeń.
6. Demo
    a. demonstracja co najmniej 3 scenariuszy:
        i. poprawny scenariusz,
        ii. scenariusz błędu/problemu sieciowego,
        iii. scenariusz związany z bezpieczeństwem (np. nieautoryzowany dostęp odrzucony).

Kryteria oceniania – ETAP 3 (45 pkt)

A. Poprawność implementacji protokołu (10 pkt)
    a. 0–3 pkt: częściowo działa / niespójne z dokumentacją
    b. 4–7 pkt: działa poprawnie w głównych scenariuszach
    c. 8–10 pkt: pełna, spójna implementacja z dobrą walidacją i obsługą stanów

B. Funkcjonalność aplikacji (10 pkt)
    a. 0–3 pkt: aplikacja nie realizuje kluczowych funkcji
    b. 4–7 pkt: realizuje podstawowe funkcje
    c. 8–10 pkt: funkcje kompletne i zgodne z przypadkami użycia
C. Implementacja bezpieczeństwa (8 pkt)
    a. 0–2 pkt: deklarowane mechanizmy niezaimplementowane / pozorne
    b. 3–5 pkt: bezpieczeństwo częściowo działa
    c. 6–8 pkt: sensownie wdrożone mechanizmy i dobre praktyki
D. Obsługa błędów i odporność na problemy połączenia (7 pkt)
    a. 0–2 pkt: awarie powodują zawieszenie / brak reakcji
    b. 3–5 pkt: podstawowa obsługa błędów działa
    c. 6–7 pkt: dobra odporność, sensowne timeouty, recovery i logowanie
E. Testy i weryfikacja działania (5 pkt)
    a. 0–1 pkt: brak testów / tylko demo “na żywo”
    b. 2–3 pkt: podstawowe testy
    c. 4–5 pkt: testy funkcjonalne + błędów, wyniki opisane
F. Jakość kodu i dokumentacja techniczna (5 pkt)
    a. 0–1 pkt: kod trudny do oceny, brak instrukcji uruchomienia
    b. 2–3 pkt: kod poprawny, dokumentacja minimalna
    c. 4–5 pkt: czytelny kod, struktura projektu, instrukcja uruchomienia i opis ograniczeń

Ocena z projektu grupowego:

- Etap 1: 30 pkt
- Etap 2: 25 pkt
- Etap 3: 45 pkt
- Razem: 100 pkt

Uwaga! warunkiem oceniania kolejnego etapu jest uzyskanie minimum 50% pkt etapu poprzedzającego.

Progi ocen:

- <50 pkt → 2.0 (niedostateczny)
- 50 → 3.0
- 60 → 3.5
- 70 → 4.0
- 80 → 4.5
- 90 → 5.0

Punkty bonusowe

Za wyjątkowo dobre przygotowanie niektórych elementów projektu możliwość uzyskania do +10 pkt bonusowych (lub podniesienie oceny o pół stopnia).

Punkty bonusowe można dostać za przygotowanie wyjątkowo dobrej:

- dokumentacji (wyjątkowo czytelna, dobrze opisana, schludnie zrobiona),
- testów automatycznych,

- konteneryzację (Docker),
- CI/CD,
- monitoring/logowanie,
- analizę wydajności,
- lub inne ciekawe rozwiązania

Postanowienia końcowe

W projektach grupowych może się zdarzyć sytuacja (mam nadzieję, że się nie wydarzy), że jedna osoba będzie robić sumiennie swoją rzecz, a druga nie. Proszę wtedy zgłosić, wtedy w takiej grupie oceny będą indywidualne wg tego ile dany student zrobił z części, którą miał się zająć.